

Practical 1

1.1:Design a ANN base system for the classification of the apple having two feature colors and weight (Binary Classification).

```
import numpy as np

# data ---> { apple colour , apple weight in gm}

# green ---> 0

# red ---> 1

# yellow ---> 2

X = np.array([
    [0, 120],
    [0, 130],
    [1, 140],
    [1, 150],
    [2, 160],
    [2, 170]
])

#Label

y = np.array([0, 0, 0, 1, 1, 1])

def step_function(x):
    return 1 if x >= 0 else 0

weights = np.random.randn(2)

bias = 0.5

learning_rate = 0.05

epochs = 120

for _ in range(epochs):
```

```

for i in range(len(X)):
    linear_output = np.dot(X[i], weights) + bias
    y_pred = step_function(linear_output)
    error = y[i] - y_pred
    weights += learning_rate * error * X[i]
    bias += learning_rate * error

print("Final Weights:", weights)
print("Final Bias:", bias)
def predict_apple(new_fruit):
    linear_output = np.dot(new_fruit, weights) + bias
    y_pred = step_function(linear_output)
    return "Apple" if y_pred == 1 else "Not apple"

apple = np.array([150, 8])
print(predict_apple(apple))

```

1.2: Design a ANN base system for the classification of the apple having two feature colors and weight (Binary Classification).(normalization)

```

import numpy as np

class Perceptron:

    def __init__(self, learning_rate=0.05, epochs=120):
        self.eta = learning_rate
        self.epochs = epochs
        self.weights = np.random.randn(2)
        self.bias = 0.5

    def sigmoid(self, x):
        return 1 / (1 + np.exp(-x))

    def fit(self, X, y):
        for _ in range(self.epochs):
            for i in range(len(X)):
                linear_output = np.dot(X[i], self.weights) + self.bias
                y_pred = self.sigmoid(linear_output)

                error = y[i] - y_pred
                self.weights += self.eta * error * X[i]
                self.bias += self.eta * error
            return self

    def predict(self, new_fruit):
        linear_output = np.dot(new_fruit, self.weights) + self.bias
        y_pred = self.sigmoid(linear_output)
        return "Apple" if y_pred >= 0.5 else "Not apple"

X = np.array([
    [0, 120/200], [0, 130/200], [1, 140/200],

```

```
[1, 150/200], [2, 160/200], [2, 170/200]
])

# 0 = Not apple & 1 = apple
y = np.array([0, 0, 0, 1, 1, 1])

model = Perceptron(learning_rate=0.1, epochs=500)
model.fit(X, y)

test_fruit = np.array([2, 150/200])
print(f"Prediction: {model.predict(test_fruit)}")
```

1.3: Design a ANN base system for the classification of the apple having two feature colors and weight (Binary Classification) WITH BACKPROPAGATION NETWORK

PRACTICAL-2

2.1: Demonstrate a very simple RNN that takes 2 sequences of numbers (1 feature per time step) and produces a single numeric output without any training.

```
import numpy as np  
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import SimpleRNN, Dense
```

1. Input sequences (2 sequences(hidden nodes), 3-time steps each, 1 feature)

```
X = np.array([
    [[1], [2], [3]],          sequence 1
    [[2], [3], [4]]          sequence 2
])
```

2. Build RNN model

```
model = Sequential()
model.add(SimpleRNN(2, input_shape=(3, 1)))    2 hidden neurons
model.add(Dense(1))                            output layer
```

3. No compilation or training — just forward pass

```
predictions = model.predict(X)
```

4. Print predictions (random because weights are not trained)

```
print("Input sequences:\n", X.reshape(2, 3))
print("Predicted outputs (random):\n", predictions)
```

2.2: Train a very simple RNN that takes 2 sequences of numbers (1 feature per time step) and produces a single numeric output without any training.

Import libraries

```
import numpy as np
```

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import SimpleRNN, Dense
```

1. Input sequences (2 sequences, 3-time steps, 1 feature)

```
X = np.array([
    [[1], [2], [3]],
    [[2], [3], [4]]
])
```

2. Target outputs

```
y = np.array([
    [6],                target for sequence 1
    [9]                target for sequence 2
])
```

3. Build RNN model

```
model = Sequential()
model.add(SimpleRNN(2, input_shape=(3, 1)))  2 hidden neurons
model.add(Dense(1))                          output neuron
```

4. Compile model (training setup)

```
model.compile(
    optimizer='adam',  updates weights
    loss='mse'         measures error
)
```

5. Train model

```
model.fit(X, y, epochs=500, verbose=0)
```

6. Test predictions

```
predictions = model.predict(X)
```

```
print("Predictions after training:")
```

```
print(predictions)
```

2.3 Predict the next number in a sequence (sequence learning).

```
import numpy as np
```

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import SimpleRNN, Dense
```

```
# Input sequences (X) and targets (y)
```

```
X = np.array([
```

```
    [1, 2, 3],
```



```
[2, 3, 4],  
[3, 4, 5],  
[4, 5, 6]  
])
```

```
y = np.array([4, 5, 6, 7])
```

```
# Reshape for RNN: (samples, timesteps, features)
```

```
X = X.reshape((X.shape[0], X.shape[1], 1))
```

```
model = Sequential()
```

```
model.add(SimpleRNN(10, activation='tanh', input_shape=(3, 1)))
```

```
model.add(Dense(1))
```

```
model.compile(
```

```
    optimizer='adam',
```

```
    loss='mean_squared_error'
```

```
)
```

```
model.fit(X, y, epochs=200, verbose=1)
```

```
# Test input
```

```
test_input = np.array([[5, 6, 7]])
```

```
test_input = test_input.reshape((1, 3, 1))
```

```
prediction = model.predict(test_input)
print("Predicted value:", prediction)
```

2.4 SENTIMENT ANALYSIS FOR THREE MOVIE REVIEWS

Movie was good

Movie was bad

Movie is not good

gIn two classes [0,1]